

PROJET BASE DE DONNEES



GAMEHUB

MongoDB · PostgreSQL
Redis · Neo4J

Présenté par :
Madec Eliott
Hacher David

Encadrant :
Dreyfus John

Table des matières

Introduction	4
2. Présentation générale du projet.....	5
2.1 Présentation de l'application.....	5
2.2 Fonctionnalités principales.....	5
2.3 Objectifs techniques	5
2.4 Répartition générale des données.....	6
2.5 Choix du sujet.....	6
3. Architecture générale	7
3.1 Principe général	7
3.2 Architecture globale.....	7
3.3 Répartition des responsabilités.....	8
PostgreSQL.....	8
MongoDB	8
Redis	9
Neo4j	9
3.4 Infrastructure Docker	9
3.5 Justification de l'architecture.....	10
4. Modélisation de la base PostgreSQL	10
4.1 Pourquoi PostgreSQL ?	10
4.2 Données stockées	10
4.3 Modèle relationnel	11
4.4 Schéma relationnel	12
4.5 Justification du modèle.....	13
4.6 Exemple de requête.....	13
4.7 Bilan	13
5. Modélisation de la base MongoDB	14
5.1 Pourquoi MongoDB ?	14
5.2 Données stockées	14
5.3 Modèle documentaire	15
5.4 Justification du modèle.....	16
5.5 Exemple de requête.....	16
5.6 Comparaison avec PostgreSQL.....	18
5.7 Bilan	18

6. Modélisation de la base Redis	18
6.1 Pourquoi Redis ?	18
6.2 Données stockées	19
6.3 Modèle de données.....	20
6.4 Justification du modèle.....	20
6.5 Exemple de commandes.....	21
6.6 Comparaison avec PostgreSQL.....	21
6.7 Bilan	22
7. Modélisation de la base Neo4j.....	22
7.1 Pourquoi Neo4j ?	22
7.2 Données stockées	22
7.3 Relations utilisées	23
7.4 Modèle de graphe	23
7.5 Justification du modèle.....	24
7.6 Exemples de requêtes Cypher	24
7.7 Bilan	25
8. Tableau de répartition des données.....	26
8.1 Répartition des responsabilités.....	26
8.2 Gestion des redondances.....	27
8.3 Justification de l'architecture polyglotte	27
9. Requêtes représentatives.....	27
9.1 PostgreSQL	28
9.2 MongoDB.....	28
9.3 Redis	29
9.4 Neo4j.....	29
9.5 Bilan	30
10. Conclusion.....	30
Conclusion personnelle	31

Introduction

Dans le cadre du cours de bases de données NoSQL, ce projet a pour objectif de mettre en place une architecture de **persistance polyglotte** autour d'une même application. La persistance polyglotte consiste à utiliser plusieurs systèmes de bases de données au sein d'un même projet, chaque technologie étant choisie selon le type de données à stocker et les besoins de l'application.

Le projet retenu est **GameHub**, une application simulée de gestion de bibliothèque de jeux vidéo multi-plateformes. L'objectif de GameHub est de permettre à un utilisateur de consulter les jeux qu'il possède, quelle que soit leur plateforme d'origine simulée, comme Steam, Epic Games, GOG, PlayStation ou Xbox. L'utilisateur peut également suivre sa progression, consulter ses achievements, visualiser un classement, suivre une activité récente et obtenir des recommandations basées sur les jeux possédés par ses amis.

Le projet ne se connecte pas réellement aux plateformes externes comme Steam ou Epic Games. Les données sont entièrement simulées et chargées automatiquement grâce à des scripts d'initialisation. Ce choix permet de concentrer le travail sur l'objectif principal du projet : la conception, la mise en place et l'exploitation de plusieurs bases de données complémentaires.

L'architecture repose sur quatre bases de données obligatoires :

- **PostgreSQL**, pour les données relationnelles structurées ;
- **MongoDB**, pour les documents liés aux achievements ;
- **Redis**, pour les données rapides, temporaires et les classements ;
- **Neo4j**, pour les relations sociales et les recommandations.

L'application reste volontairement simple. Elle sert principalement de support pour démontrer que les quatre bases sont correctement utilisées, alimentées et interrogées. Le cœur du projet porte donc sur la répartition des données, la justification des choix de modélisation, les scripts d'initialisation et les requêtes représentatives propres à chaque base.

2. Présentation générale du projet

2.1 Présentation de l'application

Le projet développé est une application nommée **GameHub**, dont l'objectif est de centraliser la bibliothèque de jeux vidéo d'un utilisateur au sein d'une seule interface.

Aujourd'hui, les joueurs utilisent plusieurs plateformes de distribution numérique telles que Steam, Epic Games, GOG ou encore les boutiques des différentes consoles. Il devient alors difficile d'obtenir une vision globale de sa bibliothèque de jeux et de suivre sa progression.

GameHub répond à cette problématique en proposant une application unique permettant de consulter l'ensemble des jeux possédés, indépendamment de leur plateforme d'origine. Dans le cadre de ce projet, les plateformes sont entièrement simulées et aucune communication avec des services externes n'est réalisée.

L'application constitue principalement un support permettant de mettre en œuvre plusieurs systèmes de bases de données au sein d'une même architecture.

2.2 Fonctionnalités principales

L'application permet à un utilisateur de :

- Consulter la liste de ses jeux vidéo ;
- Connaître la plateforme sur laquelle chaque jeu est disponible ;
- Visualiser son temps de jeu et son pourcentage de progression ;
- Consulter les achievements disponibles ainsi que ceux déjà débloqués ;
- Afficher un classement des joueurs selon leur progression moyenne ;
- Consulter les dernières activités des utilisateurs ;
- Visualiser sa liste d'amis ;
- Obtenir des recommandations de jeux en fonction des bibliothèques de ses amis.

Ces fonctionnalités sont volontairement simples afin de mettre en avant le fonctionnement des différentes bases de données plutôt que le développement de l'application elle-même.

2.3 Objectifs techniques

Le projet poursuit plusieurs objectifs techniques.

Le premier consiste à démontrer qu'il est possible d'utiliser plusieurs systèmes de gestion de bases de données au sein d'une même application.

Le second est de répartir les données selon les points forts de chaque technologie afin d'obtenir une architecture cohérente.

Enfin, l'ensemble de l'infrastructure doit être entièrement reproductible grâce à Docker. Une simple commande permet de lancer les quatre bases de données, de créer automatiquement les structures nécessaires et de charger un jeu de données représentatif.

2.4 Répartition générale des données

Les données de l'application sont réparties entre quatre bases de données spécialisées.

- **PostgreSQL** stocke les données fortement structurées de l'application, comme les utilisateurs, les jeux, les plateformes et les bibliothèques.
- **MongoDB** est utilisée pour stocker les achievements des jeux ainsi que ceux débloqués par les utilisateurs sous forme de documents.
- **Redis** conserve les informations nécessitant un accès très rapide, notamment le classement des joueurs, les activités récentes et le statut en ligne des utilisateurs.
- **Neo4j** modélise les relations entre les utilisateurs ainsi que les jeux possédés afin de proposer des recommandations basées sur le réseau d'amis.

Cette répartition permet d'exploiter les avantages de chaque technologie plutôt que de chercher à tout stocker dans une seule base de données.

2.5 Choix du sujet

Le choix du thème des jeux vidéo a été motivé par la richesse des données manipulées.

Une bibliothèque de jeux regroupe naturellement des informations relationnelles (utilisateurs, jeux, plateformes), des documents complexes (achievements), des données temporaires (statuts en ligne, classements) ainsi que des relations sociales (amis, recommandations).

Ce contexte constitue donc un excellent cas d'étude pour illustrer le principe de la **persistance polyglotte**, dans lequel plusieurs bases de données collaborent au sein d'une même application.

3. Architecture générale

3.1 Principe général

L'architecture de **GameHub** repose sur le principe de la **persistance polyglotte**, qui consiste à utiliser plusieurs systèmes de gestion de bases de données au sein d'une même application. Plutôt que de centraliser toutes les données dans une seule base, chaque technologie est utilisée pour le type de données qu'elle gère le plus efficacement.

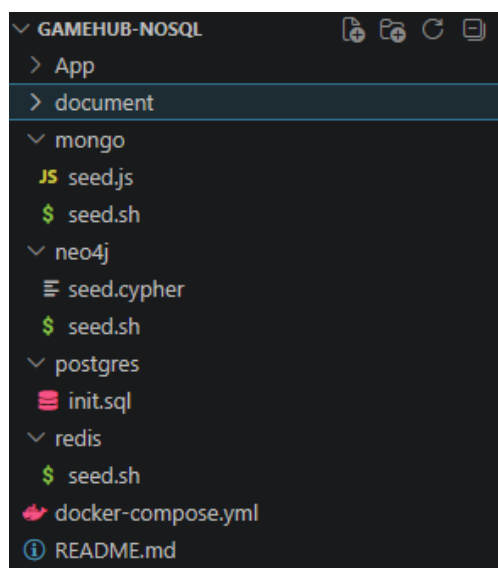
L'application joue le rôle d'intermédiaire entre l'utilisateur et les différentes bases de données. Selon la fonctionnalité demandée, elle interroge la base de données la plus adaptée puis restitue les informations à l'utilisateur.

Cette approche permet d'obtenir une architecture plus performante, plus évolutive et plus adaptée aux différents types de données manipulées.

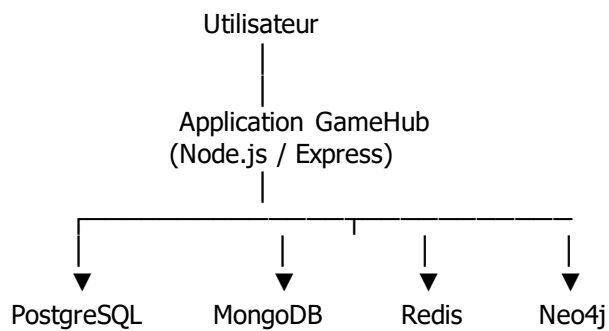
3.2 Architecture globale

L'ensemble de l'infrastructure est entièrement conteneurisé grâce à **Docker Compose**. Chaque base de données est exécutée dans son propre conteneur Docker, accompagné d'un script d'initialisation permettant de créer automatiquement les données nécessaires au premier lancement.

L'application communique directement avec les quatre bases de données.



Le schéma général de l'architecture est le suivant :



Chaque base possède une responsabilité bien définie afin d'éviter les redondances inutiles.

3.3 Répartition des responsabilités

Les responsabilités ont été réparties selon les caractéristiques de chaque technologie.

PostgreSQL

PostgreSQL constitue la base de données principale de l'application.

Elle stocke les données relationnelles fortement structurées :

- Les utilisateurs ;
- Les jeux ;
- Les plateformes ;
- Les bibliothèques des utilisateurs ;
- Les temps de jeu ;
- Les pourcentages de progression.

Cette base garantit l'intégrité des données grâce aux clés primaires, aux clés étrangères et aux contraintes relationnelles.

MongoDB

MongoDB est utilisée pour stocker les achievements.

Chaque jeu possède un document contenant l'ensemble de ses achievements. Les achievements débloqués par les utilisateurs sont également stockés sous forme de documents.

Cette structure permet de regrouper des informations de taille variable dans un même document, sans imposer un schéma relationnel complexe.

Redis

Redis est utilisé pour gérer les informations nécessitant un accès extrêmement rapide.

Dans notre projet, Redis stocke :

- Le classement des joueurs ;
- Les dernières activités ;
- Le statut en ligne des utilisateurs.

Certaines informations utilisent un **TTL (Time To Live)** afin d'être automatiquement supprimées après une durée déterminée.

Neo4j

Neo4j est dédié aux données fortement liées entre elles.

Il modélise :

- Les relations d'amitié ;
- Les jeux possédés par chaque utilisateur.

Cette représentation sous forme de graphe permet d'effectuer facilement des parcours de relations afin de proposer des recommandations de jeux basées sur les bibliothèques des amis.

3.4 Infrastructure Docker

L'ensemble de l'environnement est déployé grâce à un unique fichier **docker-compose.yml**.

Le lancement du projet s'effectue à l'aide de la commande :

```
docker compose up --build
```

Cette commande réalise automatiquement les opérations suivantes :

1. Démarrage des quatre bases de données ;
2. Création des structures de données ;
3. Exécution des scripts d'initialisation (seed) ;
4. Chargement du jeu de données de démonstration ;
5. Démarrage de l'application.

Ainsi, aucune manipulation manuelle n'est nécessaire après le lancement de l'infrastructure.

3.5 Justification de l'architecture

Cette architecture répond directement aux objectifs du projet en démontrant l'intérêt de la persistance polyglotte.

Chaque base de données est utilisée pour le domaine dans lequel elle est la plus performante :

- **PostgreSQL** assure la cohérence des données relationnelles.
- **MongoDB** facilite le stockage de documents complexes et évolutifs.
- **Redis** fournit un accès très rapide aux données temporaires.
- **Neo4j** simplifie l'exploitation des relations entre les utilisateurs.

L'utilisation conjointe de ces technologies permet d'obtenir une architecture plus adaptée qu'une solution reposant sur une seule base de données.

4. Modélisation de la base PostgreSQL

4.1 Pourquoi PostgreSQL ?

PostgreSQL est utilisé comme **base de données principale** de l'application GameHub. Son rôle est de stocker toutes les données fortement structurées et présentant de nombreuses relations entre elles.

Dans une bibliothèque de jeux vidéo, certaines informations doivent respecter des contraintes d'intégrité afin de garantir la cohérence des données. Par exemple, un utilisateur ne peut posséder qu'un jeu existant, et un jeu doit être associé à une plateforme existante.

Une base relationnelle est particulièrement adaptée à ce type de données grâce à l'utilisation de clés primaires, de clés étrangères et de contraintes d'intégrité.

Nous avons donc choisi PostgreSQL pour gérer les entités centrales de l'application.

4.2 Données stockées

La base PostgreSQL contient les informations principales de l'application :

- Les utilisateurs ;
- Les jeux ;
- Les plateformes ;
- Les bibliothèques des utilisateurs.

Elle constitue la référence principale des données du projet.

Les autres bases viennent uniquement compléter certaines fonctionnalités spécialisées.

4.3 Modèle relationnel

La base est composée de quatre tables.

Table users

Elle contient les informations des utilisateurs.

Champ	Type	Description
id	SERIAL	Identifiant unique
username	VARCHAR	Nom de l'utilisateur
email	VARCHAR	Adresse e-mail
created_at	DATE	Date d'inscription

Table games

Elle contient les jeux disponibles.

Champ	Type	Description
id	SERIAL	Identifiant du jeu
title	VARCHAR	Nom du jeu
genre	VARCHAR	Genre
publisher	VARCHAR	Éditeur
release_date	DATE	Date de sortie

Table platforms

Elle recense les plateformes de jeux.

Champ	Type	Description
id	SERIAL	Identifiant
name	VARCHAR	Nom de la plateforme

Table user_games

Cette table établit la relation entre un utilisateur et un jeu.

Elle stocke également les informations de progression.

Champ	Type	Description
id	SERIAL	Identifiant
user_id	INTEGER	Utilisateur
game_id	INTEGER	Jeu
platform_id	INTEGER	Plateforme
playtime_hours	INTEGER	Temps de jeu
completion_percent	INTEGER	Progression
added_at	DATE	Date d'ajout

Cette table possède trois clés étrangères :

- user_id
- game_id
- platform_id

Elle assure ainsi la cohérence des relations entre les différentes entités.

4.4 Schéma relationnel

Le modèle relationnel de PostgreSQL peut être résumé de la manière suivante :

users

id (PK)

username

email

created_at

games

id (PK)

title

genre

publisher

release_date

platforms

id (PK)

name

user_games

id (PK)

user_id (FK → users.id)

game_id (FK → games.id)

platform_id (FK → platforms.id)

playtime_hours

completion_percent

added_at

4.5 Justification du modèle

Le choix d'une base relationnelle s'explique par la nature des données manipulées.

Les utilisateurs, les jeux et les plateformes entretiennent de nombreuses relations. PostgreSQL garantit que ces relations restent cohérentes grâce aux clés étrangères.

Cette modélisation évite les doublons, facilite les mises à jour et permet d'effectuer des requêtes complexes à l'aide de jointures.

Elle constitue donc la base de référence de l'application.

4.6 Exemple de requête

L'une des requêtes principales permet d'afficher la bibliothèque d'un utilisateur.

```
SELECT
    u.username,
    g.title,
    p.name AS platform,
    ug.playtime_hours,
    ug.completion_percent
FROM user_games ug
JOIN users u ON ug.user_id = u.id
JOIN games g ON ug.game_id = g.id
JOIN platforms p ON ug.platform_id = p.id
ORDER BY u.username;
```

Cette requête illustre l'intérêt d'une base relationnelle, capable de combiner plusieurs tables grâce aux jointures afin de restituer une information complète.

4.7 Bilan

PostgreSQL joue un rôle central dans l'architecture de GameHub.

Il stocke les données les plus structurées et garantit leur cohérence grâce aux mécanismes relationnels. Les informations contenues dans cette base servent ensuite de référence aux autres bases de données du projet, qui apportent chacune des fonctionnalités spécialisées.

5. Modélisation de la base MongoDB

5.1 Pourquoi MongoDB ?

MongoDB est une base de données orientée documents. Contrairement aux bases relationnelles, elle stocke les informations sous forme de documents au format BSON (Binary JSON), ce qui permet de regrouper dans un même objet plusieurs informations liées entre elles.

Dans le cadre de GameHub, MongoDB est utilisée pour gérer les **achievements** (succès) des jeux vidéo ainsi que leur progression par les utilisateurs.

Les achievements représentent un bon exemple de données dont la structure peut varier d'un jeu à l'autre. Certains jeux possèdent quelques succès, tandis que d'autres en contiennent plusieurs centaines. De plus, chaque achievement possède des caractéristiques différentes (description, difficulté, rareté, récompense, etc.). Une base documentaire est donc plus adaptée qu'un modèle relationnel pour représenter ce type d'informations.

Le choix de MongoDB permet ainsi de conserver des documents complets, facilement extensibles et plus proches de la structure naturelle des données.

5.2 Données stockées

Deux collections principales sont utilisées.

Collection achievements

Cette collection contient la définition des achievements disponibles pour chaque jeu.

Chaque document représente un jeu et regroupe l'ensemble de ses achievements.

Exemple simplifié :

```
{
  "_id": 1,
  "game_id": 1,
  "game": "Cyberpunk 2077",
  "achievements": [
    {
      "name": "Premier Pas",
      "description": "Terminer le prologue",
      "points": 10
    },
    {
      "name": "Mercenaire",
      "description": "Compléter 50 missions",
      "points": 50
    }
  ]
}
```


Cette organisation évite de créer plusieurs tables comme cela aurait été nécessaire dans une base relationnelle.

Collection **user_achievements**

Cette collection stocke les achievements débloqués par chaque utilisateur.

Chaque document contient :

- L'identifiant de l'utilisateur ;
- L'identifiant du jeu ;
- La liste des achievements déjà obtenus ;
- La date de déblocage.

Exemple simplifié :

```
{
  "_id": 1,
  "user_id": 1,
  "game_id": 1,
  "unlocked": [
    {
      "achievement": "Premier Pas",
      "date": "2026-01-15"
    }
  ]
}
```

Cette approche permet de conserver toutes les informations relatives à un utilisateur dans un seul document.

5.3 Modèle documentaire

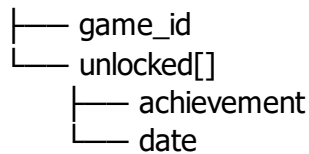
Le modèle retenu repose sur deux collections :

achievements

- game_id
- game
- achievements[]
 - name
 - description
 - points

user_achievements

- user_id



Les tableaux (achievements[] et unlocked[]) illustrent parfaitement l'un des principaux avantages de MongoDB : la possibilité d'intégrer directement plusieurs objets dans un même document.

5.4 Justification du modèle

Le modèle documentaire présente plusieurs avantages dans ce contexte.

Les achievements d'un jeu sont naturellement liés entre eux et sont presque toujours consultés ensemble. Les stocker dans un même document réduit le nombre de requêtes nécessaires et simplifie la lecture des données.

De plus, les jeux ne possèdent pas tous le même nombre de succès. Certains peuvent contenir seulement quelques achievements, tandis que d'autres en proposent plusieurs dizaines. MongoDB permet cette variabilité sans modifier la structure de la base.

Enfin, ce modèle est facilement extensible. Il serait possible d'ajouter de nouvelles propriétés (niveau de difficulté, icône, rareté, récompense, etc.) sans modifier les documents existants.

5.5 Exemple de requête

Afficher les achievements du jeu *Cyberpunk 2077* :

```
db.achievements.find(
  { game_id: 1 },
  {
    _id: 0,
    game: 1,
    achievements: 1
  }
)
```

Cette requête retourne directement le document contenant tous les achievements du jeu.

Afficher les achievements débloqués par un utilisateur :

```
use gamehub
```

```
db.achievements.find().pretty()use gamehub
```

`db.achievements.find().pretty()` Cette requête permet d'obtenir rapidement la progression d'un utilisateur pour chacun de ses jeux.

```

gamehub> db.achievements.find().pretty()
[
  {
    _id: ObjectId('6a4ccb1f90b5e9b3cb75bc17'),
    gameId: 1,
    gameTitle: 'Cyberpunk 2077',
    achievements: [
      {
        code: 'cp_first_contract',
        name: 'Premier contrat',
        description: 'Terminer sa première mission',
        points: 10,
        rarity: 'common'
      },
      {
        code: 'cp_night_city',
        name: 'Bienvenue à Night City',
        description: 'Découvrir la ville principale',
        points: 20,
        rarity: 'common'
      },
      {
        code: 'cp_act1',
        name: "Finir l'acte 1",
        description: "Terminer le premier acte de l'histoire",
        points: 40,
        rarity: 'rare'
      },
      {
        code: 'cp_legend',
        name: 'Légende de Night City',
        description: "Terminer l'histoire principale",
        points: 100,
        rarity: 'legendary'
      }
    ]
  },
  {
    _id: ObjectId('6a4ccb1f90b5e9b3cb75bc18'),
    gameId: 2,
    gameTitle: 'Hades',
    achievements: [
      {
        code: 'hades_escape',
        name: 'Première évasion',
        description: "Réussir à s'échapper une première fois",
        points: 50,
        rarity: 'rare'
      }
    ]
  }
]

```

5.6 Comparaison avec PostgreSQL

Une implémentation relationnelle aurait nécessité plusieurs tables supplémentaires :

- achievements ;
- user_achievements ;
- achievement_types ;
- Éventuellement des tables d'association.

Le nombre de jointures aurait rapidement augmenté avec la complexité des données.

Avec MongoDB, les informations sont regroupées dans des documents cohérents, directement exploitables par l'application, ce qui simplifie la modélisation et réduit le nombre de requêtes.

5.7 Bilan

MongoDB est utilisée pour stocker les achievements des jeux et leur progression chez les utilisateurs.

Son modèle documentaire permet de représenter naturellement des données hiérarchiques et évolutives, sans imposer un schéma relationnel complexe. Cette approche offre une grande souplesse tout en restant cohérente avec les besoins de l'application.

6. Modélisation de la base Redis

6.1 Pourquoi Redis ?

Redis est une base de données **clé-valeur entièrement stockée en mémoire**.

Contrairement aux bases relationnelles ou documentaires, elle est conçue pour fournir un accès extrêmement rapide aux données.

Dans le projet GameHub, Redis est utilisé pour stocker des informations qui nécessitent des lectures très fréquentes mais qui ne sont pas destinées à être conservées de manière permanente.

Ces données concernent principalement :

- Le classement des joueurs ;
- Les activités récentes ;
- Le statut de connexion des utilisateurs.

Ce sont des informations qui évoluent régulièrement et qui doivent être consultées rapidement par l'application.

Le choix de Redis permet ainsi d'obtenir d'excellentes performances tout en limitant la charge des autres bases de données.

6.2 Données stockées

Trois structures Redis sont utilisées.

Leaderboard

Le classement des joueurs est stocké dans un **Sorted Set**.

Chaque utilisateur possède un score correspondant à son pourcentage moyen de progression.

Exemple :

leaderboard:completion

Sarah -> 83
David -> 76
Lucas -> 70
Emma -> 70

Le Sorted Set permet de récupérer instantanément les joueurs les mieux classés.

Activités récentes

Les dernières actions réalisées par les utilisateurs sont stockées dans une **List**.

Exemple :

recent_activity

Emma added Rocket League
Sarah completed Hades
Lucas defeated a boss in Elden Ring
David completed Cyberpunk 2077

Chaque nouvelle activité est ajoutée en tête de liste.

Cette structure est parfaitement adaptée à un flux chronologique.

Statut des utilisateurs

Le statut de connexion est enregistré grâce à des clés simples.

Exemple :

user:1:status → online
user:2:status → online
user:3:status → offline
user:4:status → online

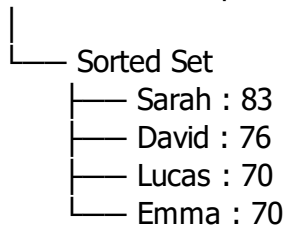
Chaque statut possède une durée de vie limitée grâce au mécanisme de **TTL (Time To Live)**.

Après expiration, Redis supprime automatiquement la clé.

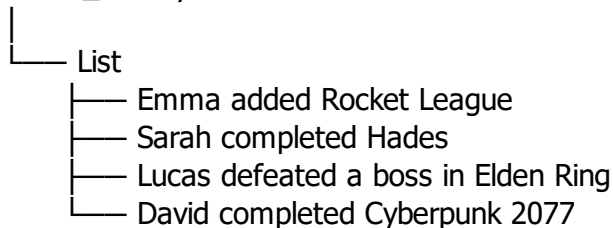
6.3 Modèle de données

Les structures utilisées sont les suivantes :

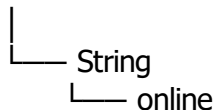
leaderboard:completion



recent_activity



user:1:status



Chaque structure est choisie selon le type d'information manipulée.

6.4 Justification du modèle

Redis propose plusieurs structures de données spécialisées.

Dans ce projet, chacune est utilisée pour répondre à un besoin précis.

Les **Sorted Sets** permettent de maintenir automatiquement un classement trié selon un score. Ils sont particulièrement adaptés aux tableaux de scores ou aux classements de joueurs.

Les **Lists** facilitent la gestion d'un historique d'événements en conservant un ordre chronologique.

Les **Strings** associées à un **TTL** sont utilisés pour représenter les statuts temporaires des utilisateurs. Lorsqu'un utilisateur devient inactif, son statut expire automatiquement sans intervention de l'application.

Cette approche limite le volume de données persistantes et améliore les performances globales.

6.5 Exemple de commandes

Afficher le classement :

```
ZREVRANGE leaderboard:completion 0 -1 WITHSCORES
```

Résultat :

```
Sarah 83
David 76
Lucas 70
Emma 70
```

Afficher les dernières activités :

```
LRANGE recent_activity 0 -1
```

```
127.0.0.1:6379> LRANGE recent_activity 0 -1
1) "Emma added Rocket League"
2) "Sarah completed Hades"
3) "Lucas defeated a boss in Elden Ring"
4) "David completed Cyberpunk 2077"
```

Afficher le statut d'un utilisateur :

```
GET user:1:status
```

```
127.0.0.1:6379> GET user:1:status
"online"
```

Afficher le temps restant avant expiration :

```
TTL user:1:status
```

```
127.0.0.1:6379> TTL user:1:status
(integer) -2
127.0.0.1:6379> SET user:1:status online EX 300
OK
127.0.0.1:6379> TTL user:1:status
(integer) 295
```

Cette dernière commande montre le fonctionnement du TTL, l'une des fonctionnalités principales de Redis. (Ici nous avons du reset le compteur car la clé avait été supprimé une fois les 5 minutes écoulés)

6.6 Comparaison avec PostgreSQL

Il serait possible de stocker ces informations dans PostgreSQL.

Cependant, cela présenterait plusieurs inconvénients.

Le classement devrait être recalculé régulièrement à partir des données relationnelles.

Les activités récentes nécessiteraient de nombreuses écritures et suppressions.

Enfin, la gestion automatique de l'expiration des statuts n'existe pas nativement dans une base relationnelle.

Redis apporte donc une solution beaucoup plus simple et beaucoup plus performante pour ce type de données.

6.7 Bilan

Redis est utilisé pour gérer les données nécessitant un accès très rapide ou une durée de vie limitée.

Grâce à ses structures spécialisées (Sorted Set, List et String avec TTL), il répond parfaitement aux besoins de GameHub en matière de performances et de gestion de données temporaires.

Son utilisation permet de décharger la base relationnelle tout en offrant une excellente réactivité à l'application.

7. Modélisation de la base Neo4j

7.1 Pourquoi Neo4j ?

Neo4j est une base de données orientée graphe. Elle est conçue pour représenter des données fortement reliées entre elles et pour effectuer facilement des parcours de relations.

Dans le projet GameHub, Neo4j est utilisée pour gérer la partie sociale de l'application :

- Les relations d'amitié entre utilisateurs ;
- Les jeux possédés par les utilisateurs ;
- Les recommandations de jeux basées sur les amis.

Ce choix est pertinent car les recommandations reposent directement sur des relations. Par exemple, pour recommander un jeu à David, on recherche les jeux possédés par ses amis mais absents de sa propre bibliothèque.

7.2 Données stockées

Neo4j contient deux types de nœuds principaux.

Nœuds User

Ils représentent les utilisateurs.

Exemple :

```
(:User {id: 1, name: "David"})
```

Nœuds Game

Ils représentent les jeux.

Exemple :

```
(:Game {id: 1, title: "Cyberpunk 2077"})
```

7.3 Relations utilisées

Deux relations principales sont utilisées.

Relation FRIEND

Elle relie deux utilisateurs amis.

```
(:User)-[:FRIEND]->(:User)
```

Exemple :

```
(:User {name: "David"})-[:FRIEND]->(:User {name: "Lucas"})
```

Relation OWNS

Elle relie un utilisateur à un jeu qu'il possède.

```
(:User)-[:OWNS]->(:Game)
```

Exemple :

```
(:User {name: "David"})-[:OWNS]->(:Game {title: "Cyberpunk 2077"})
```

7.4 Modèle de graphe

Le modèle peut être résumé ainsi :

```
(User)-[:FRIEND]->(User)
```

```
(User)-[:OWNS]->(Game)
```

Exemple simplifié :

David — FRIEND —> Lucas

David — FRIEND —> Sarah

David — OWNS —> Cyberpunk 2077

Lucas — OWNS —> Elden Ring

Sarah — OWNS —> Hollow Knight

7.5 Justification du modèle

Neo4j est particulièrement adapté à ce besoin car les fonctionnalités sociales reposent sur des relations.

Dans une base relationnelle classique, retrouver les jeux possédés par les amis d'un utilisateur nécessiterait plusieurs jointures. Avec Neo4j, cette logique s'exprime naturellement sous forme de parcours de graphe.

Le modèle retenu reste volontairement simple afin de se concentrer sur les cas d'usage principaux :

- Trouver les amis d'un utilisateur ;
- Trouver les jeux possédés par ses amis ;
- Recommander des jeux que l'utilisateur ne possède pas encore.

7.6 Exemples de requêtes Cypher

Afficher les amis de David :

```
MATCH (:User {name:"David"})-[:FRIEND]->(friend)
RETURN friend.name;
```

```
neo4j@neo4j> MATCH (:User {name:"David"})-[:FRIEND]->(friend)
              RETURN friend.name;
+-----+
| friend.name |
+-----+
| "Sarah"    |
| "Lucas"    |
+-----+
```

Afficher les jeux de David :

```
MATCH (:User {name:"David"})-[:OWNS]->(game)
RETURN game.title;
```

```
neo4j@neo4j> MATCH (:User {name: David })-[:OWNS]->(game)
              RETURN game.title;
+-----+
| game.title |
+-----+
| "The Witcher 3" |
| "Hades"        |
| "Cyberpunk 2077" |
+-----+
```

Afficher les jeux possédés par les amis de David :

```
MATCH (:User {name:"David"})-[:FRIEND]->(friend)-[:OWNS]->(game)
RETURN friend.name, game.title;
```

```
neo4j@neo4j> MATCH (:User {name:"David"})-[:FRIEND]->(friend)-[:OWNS]->(game)
                RETURN friend.name, game.title;
```

friend.name	game.title
"Sarah"	"Hollow Knight"
"Sarah"	"Stardew Valley"
"Sarah"	"Hades"
"Lucas"	"Baldur's Gate 3"
"Lucas"	"Cyberpunk 2077"
"Lucas"	"Elden Ring"

Recommander des jeux à David :

```
MATCH (me:User {name:"David"})-[:FRIEND]->(friend)-[:OWNS]->(game)
WHERE NOT (me)-[:OWNS]->(game)
RETURN DISTINCT game.title AS Recommendation;
```

```
neo4j@neo4j> MATCH (me:User {name:"David"})-[:FRIEND]->(friend)-[:OWNS]->(game)
                WHERE NOT (me)-[:OWNS]->(game)
                RETURN DISTINCT game.title AS Recommendation;
```

Recommendation
"Hollow Knight"
"Stardew Valley"
"Baldur's Gate 3"
"Elden Ring"

Cette dernière requête est la plus représentative de l'intérêt de Neo4j dans le projet. Elle permet d'obtenir directement les jeux possédés par les amis de David, mais absents de sa propre bibliothèque.

7.7 Bilan

Neo4j complète l'architecture de GameHub en apportant une modélisation orientée relations.

Elle permet de représenter simplement les liens sociaux entre utilisateurs et d'exploiter ces liens pour générer des recommandations. Cette base n'est donc pas utilisée comme un simple stockage secondaire, mais bien pour sa capacité à parcourir efficacement un graphe de relations.

8. Tableau de répartition des données

L'un des objectifs principaux du projet est de répartir intelligemment les données entre plusieurs systèmes de gestion de bases de données. Chaque technologie est utilisée en fonction de ses caractéristiques afin d'exploiter au mieux ses points forts.

Le tableau ci-dessous présente les différentes catégories de données de l'application ainsi que la base de données retenue pour leur stockage.

Donnée	Base utilisée	Justification
Utilisateurs	PostgreSQL	Données fortement structurées nécessitant une intégrité référentielle.
Jeux	PostgreSQL	Entités centrales de l'application avec relations vers les utilisateurs et les plateformes.
Plateformes	PostgreSQL	Liste fixe de plateformes nécessitant une gestion relationnelle.
Bibliothèque des utilisateurs	PostgreSQL	Association entre utilisateurs et jeux avec temps de jeu et progression.
Achievements disponibles	MongoDB	Structure documentaire permettant de regrouper les succès d'un jeu dans un même document.
Achievements débloqués	MongoDB	Documents contenant la progression d'un utilisateur pour chaque jeu.
Classement des joueurs	Redis	Données fréquemment consultées nécessitant un accès très rapide grâce aux Sorted Sets.
Activités récentes	Redis	Historique des dernières actions, naturellement représenté par une List.
Statut des utilisateurs	Redis	Données temporaires utilisant un mécanisme d'expiration automatique (TTL).
Relations d'amitié	Neo4j	Les liens entre utilisateurs sont naturellement représentés sous forme de graphe.
Jeux possédés (pour les recommandations)	Neo4j	Permet de parcourir rapidement les bibliothèques des amis afin de proposer des recommandations.

8.1 Répartition des responsabilités

Chaque base de données possède une responsabilité clairement définie.

PostgreSQL constitue la base de référence de l'application. Elle contient les données principales et garantit leur cohérence grâce aux mécanismes relationnels.

MongoDB est spécialisée dans le stockage des achievements. Son modèle documentaire permet de regrouper toutes les informations d'un jeu dans un seul document et d'accepter facilement des structures évolutives.

Redis est dédié aux données temporaires ou nécessitant des temps d'accès très faibles. Les classements, activités récentes et statuts de connexion sont ainsi accessibles instantanément.

Neo4j est utilisé exclusivement pour les données fortement reliées. Il facilite les parcours entre utilisateurs et permet de générer efficacement des recommandations de jeux.

8.2 Gestion des redondances

Certaines informations sont volontairement présentes dans plusieurs bases de données afin de répondre à des besoins différents.

Par exemple, un jeu est décrit dans **PostgreSQL** comme une entité relationnelle, tandis que **Neo4j** ne conserve que les informations nécessaires pour établir les relations entre les utilisateurs et les jeux. De la même manière, les identifiants des jeux sont utilisés dans **MongoDB** pour associer les achievements aux jeux enregistrés dans PostgreSQL.

Cette légère redondance est volontaire et maîtrisée. Elle permet à chaque base de données d'être optimisée pour son propre cas d'usage, sans remettre en cause la cohérence globale de l'application.

8.3 Justification de l'architecture polyglotte

La répartition retenue illustre le principe de **persistance polyglotte**. Plutôt que d'utiliser une seule base de données pour l'ensemble des informations, chaque technologie est exploitée dans le domaine où elle est la plus performante.

Cette architecture permet :

- De garantir la cohérence des données principales grâce à PostgreSQL ;
- De simplifier la gestion des achievements avec MongoDB ;
- D'améliorer les performances des données temporaires grâce à Redis ;
- De faciliter les parcours de relations et les recommandations avec Neo4j.

Le résultat est une architecture plus adaptée aux besoins de l'application qu'une solution reposant sur une base de données unique.

9. Requêtes représentatives

L'objectif de cette partie est de présenter quelques requêtes représentatives réalisées sur chacune des quatre bases de données. Ces requêtes illustrent les fonctionnalités principales

de chaque système de gestion de bases de données ainsi que les raisons de leur utilisation dans le projet GameHub.

9.1 PostgreSQL

PostgreSQL est utilisé pour gérer les données relationnelles de l'application. Les requêtes reposent principalement sur les jointures entre plusieurs tables.

Afficher la bibliothèque des utilisateurs

```
SELECT
    u.username,
    g.title,
    p.name AS platform,
    ug.playtime_hours,
    ug.completion_percent
FROM user_games ug
JOIN users u ON ug.user_id = u.id
JOIN games g ON ug.game_id = g.id
JOIN platforms p ON ug.platform_id = p.id
ORDER BY u.username;
```

Cette requête illustre la puissance des bases relationnelles. Les informations sont réparties dans plusieurs tables, puis regroupées grâce aux jointures afin d'obtenir une vue complète de la bibliothèque de chaque utilisateur.

9.2 MongoDB

MongoDB est utilisée pour stocker les achievements des jeux et ceux débloqués par les utilisateurs.

Afficher les achievements d'un jeu

```
db.achievements.find(
  { gameId: 1 }
)
```

Cette requête récupère le document correspondant au jeu sélectionné ainsi que l'ensemble des achievements qu'il contient.

Afficher les achievements débloqués par un utilisateur

```
db.user_achievements.find(
  { userId: 1 }
)
```

Cette requête retourne les succès débloqués par un utilisateur pour chacun de ses jeux.

Grâce au modèle documentaire, toutes les informations sont regroupées dans un seul document.

9.3 Redis

Redis est utilisé pour les données nécessitant un accès rapide.

Afficher le classement

```
ZREVRANGE leaderboard:completion 0 -1 WITHSCORES
```

Cette commande retourne les joueurs classés selon leur pourcentage moyen de progression.

L'utilisation d'un **Sorted Set** permet d'obtenir automatiquement un classement trié.

Afficher les activités récentes

```
LRANGE recent_activity 0 -1
```

Cette commande affiche les dernières actions réalisées par les utilisateurs.

Les activités sont stockées dans une **List**, parfaitement adaptée aux données chronologiques.

Vérifier le TTL d'un utilisateur

```
TTL user:1:status
```

Cette commande permet de connaître le temps restant avant la suppression automatique du statut d'un utilisateur.

Le TTL démontre la capacité de Redis à gérer des données temporaires sans intervention de l'application.

9.4 Neo4j

Neo4j est utilisé pour gérer les relations entre les utilisateurs.

Afficher les amis d'un utilisateur

```
MATCH (:User {name:"David"})-[:FRIEND]->(friend)
RETURN friend.name;
```

Cette requête parcourt directement les relations d'amitié du graphe.

Afficher les jeux possédés par les amis

```
MATCH (:User {name:"David"})-[:FRIEND]->(friend)-[:OWNS]->(game)
RETURN friend.name, game.title;
```

Cette requête illustre un parcours de plusieurs relations successives dans le graphe.

Recommander des jeux

```
MATCH (me:User {name:"David"})-[:FRIEND]->(friend)-[:OWNS]->(game)
WHERE NOT (me)-[:OWNS]->(game)
RETURN DISTINCT game.title AS Recommendation;
```

Cette requête représente le principal intérêt de Neo4j dans le projet.

Elle permet d'identifier les jeux possédés par les amis de David mais absents de sa propre bibliothèque. Les résultats obtenus constituent ainsi une liste de recommandations personnalisées.

9.5 Bilan

Les différentes requêtes présentées mettent en évidence les spécificités de chaque base de données.

- PostgreSQL exploite les relations entre les tables grâce aux jointures.
- MongoDB facilite la manipulation de documents complets contenant plusieurs informations liées.
- Redis offre un accès extrêmement rapide aux données temporaires et aux classements.
- Neo4j simplifie les parcours de relations complexes afin de générer des recommandations.

Ces exemples démontrent que chaque technologie est utilisée dans le domaine où elle est la plus performante, conformément au principe de la persistance polyglotte.

10. Conclusion

Le projet **GameHub** a permis de mettre en œuvre une architecture reposant sur le principe de la **persistance polyglotte**, consistant à utiliser plusieurs systèmes de gestion de bases de données au sein d'une même application. Plutôt que de centraliser l'ensemble des informations dans une seule base, chaque technologie a été sélectionnée en fonction de ses points forts et du type de données à gérer.

La répartition des responsabilités entre les différentes bases de données a permis d'exploiter les caractéristiques propres à chacune d'elles :

- **PostgreSQL** assure le stockage des données relationnelles et garantit leur cohérence grâce aux clés étrangères et aux contraintes d'intégrité.

- **MongoDB** permet de représenter simplement les achievements sous forme de documents flexibles et facilement extensibles.
- **Redis** fournit un accès très rapide aux données temporaires telles que les classements, les activités récentes et les statuts des utilisateurs.
- **Neo4j** facilite la gestion des relations entre utilisateurs et la génération de recommandations de jeux grâce à son modèle orienté graphe.

L'ensemble de l'infrastructure a été entièrement conteneurisé avec **Docker Compose**. Une seule commande permet de démarrer les quatre bases de données, d'exécuter automatiquement les scripts d'initialisation et de charger un jeu de données représentatif. Cette approche garantit une installation simple, reproductible et conforme aux exigences du projet.

Même si l'application développée reste volontairement simple, elle remplit pleinement son rôle de support en démontrant que les quatre bases de données sont correctement intégrées et peuvent être exploitées conjointement. Chaque technologie intervient dans un domaine précis, ce qui permet d'obtenir une architecture cohérente et adaptée aux différents besoins de l'application.

Enfin, ce projet a permis d'approfondir la compréhension des différentes familles de bases de données et de mettre en pratique leurs usages respectifs. Il montre qu'une architecture reposant sur plusieurs technologies complémentaires est souvent plus pertinente qu'une solution utilisant une unique base de données pour l'ensemble des traitements.

Conclusion personnelle

Retour d'expérience

La réalisation de ce projet nous a permis de découvrir les avantages et les spécificités des principales familles de bases de données utilisées aujourd'hui en entreprise. Nous avons appris à choisir une technologie en fonction des besoins fonctionnels plutôt que de chercher à utiliser une seule base pour toutes les données.

La mise en place de l'infrastructure Docker, l'automatisation des scripts d'initialisation ainsi que la modélisation adaptée à chaque système de gestion de bases de données nous ont permis d'acquérir une vision plus concrète de la persistance polyglotte et de son intérêt dans le développement d'applications modernes.

Nous avons également pris conscience de l'importance de la phase de conception, qui permet de justifier les choix techniques et de garantir une architecture cohérente avant même le développement de l'application.